Norm AI - Smart Attendance System

Architecture guide

1. What the system does

Norm AI is a face-recognition attendance system built for RMK Engineering College. A student opens a desktop app and looks at the webcam. The system decides whether the face is real (not a photo or a screen) and whether it belongs to that student. If both are true, attendance is recorded for the day's time slot. Teachers can review and override their students' attendance, and administrators manage holidays, time slots and the allowed Wi-Fi networks.

The design splits the work three ways so each part does what it is good at: a desktop shell that owns the camera and Wi-Fi checks, a local Python process that runs the face models, and a central API with a database that holds the records and makes the final decision.

2. Big picture

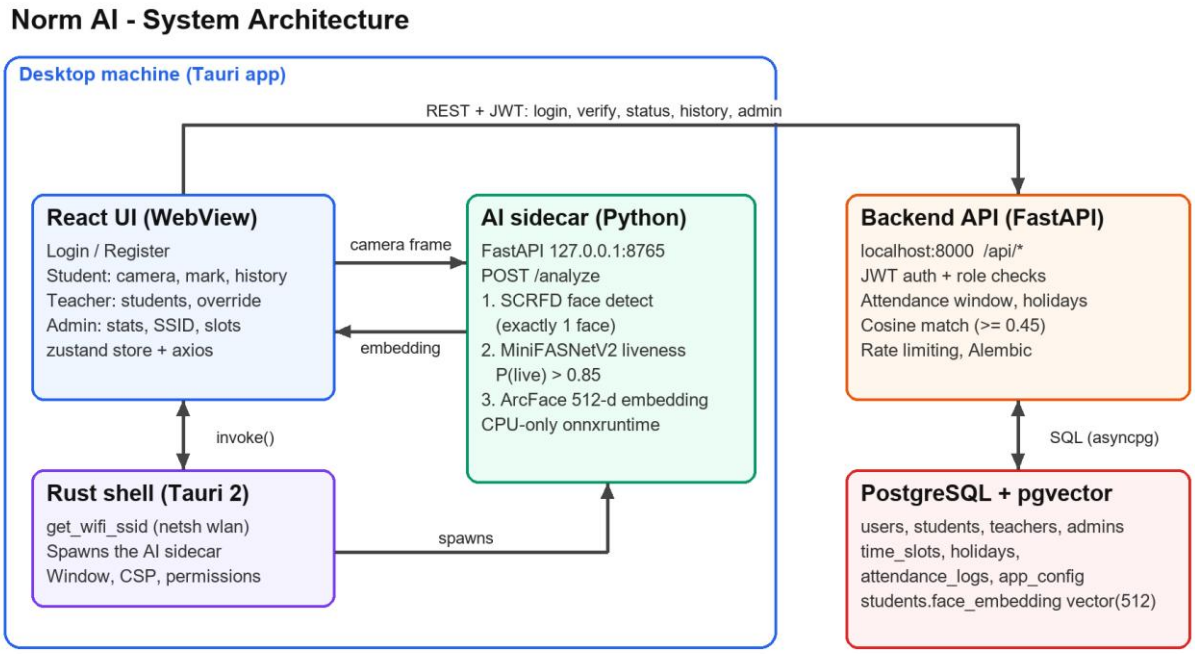


Figure 1. The four runtime pieces. Face images stay on the desktop machine; only the 512-number embedding is sent to the backend.

3. Components

Component	Technology	Responsibility	Where it runs
Frontend UI	React 18, TypeScript, Vite, Tailwind, zustand, react-router, axios	Login and registration, role dashboards (student, teacher, admin), camera capture, history and settings pages	Inside the Tauri window (dev server on port 1420)
Desktop shell	Tauri 2 (Rust)	Reads the Wi-Fi name via netsh, starts the AI sidecar, provides the native window and security policy (CSP)	User's Windows PC

Component	Technology	Responsibility	Where it runs
AI sidecar	Python, FastAPI, InsightFace buffalo_l, MiniFASNetV2 (ONNX), onnxruntime (CPU)	Detects faces, checks liveness, produces the 512-dimension face embedding	127.0.0.1:8765 (local only)
Backend API	FastAPI, SQLAlchemy 2 (async), Alembic, python-jose, passlib/bcrypt, slowapi	Authentication, role checks, time-slot and holiday rules, embedding matching, attendance records	localhost:8000 (or a server)
Database	PostgreSQL 15 with the pgvector extension	Stores users, profiles, slots, holidays, attendance and each student's face vector	Docker container, port 5432

4. Data model

Nine tables (eight application tables plus Alembic's version table), created by the migrations in norm-ai-backend/alembic/versions.

Table	Purpose
users	Login identity: email, password hash (bcrypt), role (student, teacher or admin), active flag. One row per person.
students	Profile for student users: name, register number, department, section, counsellor (a teacher), and face_embedding, a pgvector column of 512 floats.
teachers / admins	Profile rows for the other two roles (name, department and position for teachers).
time_slots	One row per working day: slot name, start time, end time. Created automatically for the day (default 08:30 to 10:00, or the admin-configured window).
holidays	Dates on which attendance is not allowed, with a reason. Sundays are treated as holidays in code.
attendance_logs	One row per student per slot per date: status (present, absent and so on), who marked it (system or teacher), similarity score, timestamp.
app_config	Key/value JSON settings: allowed Wi-Fi names and the slot window.

5. Key flows

5.1 Student enrolment

1. The student fills in the registration form and the camera captures a frame.
2. The UI posts the frame (base64 JPEG) to the sidecar at /analyze.
3. The sidecar requires exactly one face, passes the liveness check, and returns an L2-normalised 512-number embedding.
4. The UI sends the profile and embedding to POST /api/auth/register/student. The backend validates the vector (512 values, each in range), hashes the password and stores everything.

5.2 Marking attendance (the core flow)

1. The UI checks the Wi-Fi name through the Rust shell (it must be on the allowed list).
2. While the camera modal is open, the UI sends a frame to the sidecar every 2 seconds. A frame that is not live, or has zero or several faces, is ignored and the next one is tried.
3. Once a frame passes liveness (probability above 0.85), the sidecar returns the embedding and the UI calls POST /api/attendance/verify with the embedding and today's date, carrying the JWT in the header.

4. The backend applies these rules in order: not a Sunday; not a holiday; a time slot exists; the current time is inside the slot window; not already marked present (otherwise HTTP 409).
5. It then asks PostgreSQL for the cosine similarity between the stored embedding and the new one (pgvector). Below 0.45 the request is rejected with 401. Otherwise an attendance_logs row is written with the score.
6. The UI shows success, and the history page reads GET /api/attendance/history?month=YYYY-MM.

5.3 Teacher and admin actions

- Teachers: list their students and override a student's status for a slot (PUT /api/attendance/override). The backend only allows overrides for students whose counsellor is that teacher.
- Admins: dashboard statistics, add or remove holidays, generate slots, change the slot window and the allowed Wi-Fi names, and register further admins.

6. API surface

Route group	What it provides
/api/auth	register/student, register/teacher, register/admin (admin only), login, refresh
/api/students	me (get and update profile), all (teacher and admin list)
/api/attendance	verify, status (today), history (by month), override
/api/timeslots	list, today, generate (admin)
/api/holidays	list, add, delete by date
/api/teachers	me, list, and counselled-student operations
/api/admin	stats, ssid-config, slot-config, recent-activity

Every response uses the same envelope: success, data and error. Interactive documentation is served at http://localhost:8000/docs.

7. Security and trust model

- Authentication uses short-lived JWT access tokens (15 minutes) and refresh tokens (7 days). Passwords are stored as bcrypt hashes. A SECRET_KEY of at least 64 characters is required to start the backend.
- Role checks are enforced on the server by a role_required dependency, so a student token cannot reach admin routes (tested: HTTP 403).
- Login and registration are rate limited (10 requests per minute).
- The sidecar listens on 127.0.0.1 only, refuses to start if a GPU provider is present, and fails closed: if the liveness model is missing, nobody is marked live.
- Privacy: only the numeric embedding is stored; the photo itself is not sent to the backend.

Known limitation: liveness runs on the client, and the backend trusts the embedding it receives. A modified client could send an embedding without passing liveness. Closing this would need the backend (or a signed sidecar response) to verify liveness itself.

8. Configuration, running and testing

Item	Detail
Backend settings	norm-ai-backend/.env (copy .env.example): DATABASE_URL, SECRET_KEY, token lifetimes.

Item	Detail
Sidecar tuning	Environment variables LIVENESS_THRESHOLD (default 0.85) and LIVE_CLASS_INDEX (default 1).
Python versions	The backend works on Python 3.13; the sidecar's pinned packages need 3.11 (separate virtual environment).
Start order	Postgres (docker compose up -d), backend (uvicorn app.main:app), sidecar (python ai_pipeline.py), then the frontend (npm run dev or npm run tauri dev).
Default admin	admin@rmkec.ac.in with the password set in app/seed_admin.py. Change it before real use.
Tests	pytest in norm-ai-backend: 15 tests (security, schemas, API smoke, and 6 database-backed tests for duplicates, windows, holidays and wrong faces). Frontend: tsc type check and production build.
CI	.github/workflows/ci.yml runs the backend tests against a Postgres service, plus the frontend type check.

9. Verified state and open items

- Verified on this machine: database migrations, admin seed, all backend tests, sidecar model loading, a live webcam enrolment and attendance mark through the real API (similarity 0.73), duplicate rejection and role checks.
- Liveness on a real face scored about 0.9 to 0.99 on most frames with occasional dips (0.48 and 0.81 seen), which the UI absorbs by retrying every 2 seconds. Behaviour against a photo or screen replay has not been measured here.
- Not yet done: a packaged installer (the sidecar still needs a PyInstaller build), a click-through test of the UI screens, and running the Tauri window itself.
- Minor: the camera modal builds today's date from UTC. For IST users this only differs between midnight and 05:30, outside the attendance window.